1. Tasarım Sistemi Tanımları (Design System)

Önce `tailwind.config.js` ve `src/index.css` dosyalarını güncelleyerek "Derin Sinema" temasını tüm uygulamaya yayalım.

`tailwind.config.js` (Güncellenmiş)
```javascript
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,ts,jsx,tsx}"],
  theme: {
    extend: {
      colors: {
        cinema: {
          bg: "#050507",        // Derin boşluk siyahı
          surface: "#121216",   // Cam efektli yüzeyler
          surfaceHover: "#1A1A20",
          gold: "#FFB300",      // Yüksek puan, başarı, vurgu
          goldDim: "rgba(255, 179, 0, 0.15)",
          cyan: "#00E5FF",      // AI, keşif, veri görselleştirme
          cyanDim: "rgba(0, 229, 255, 0.15)",
          danger: "#FF453A",    // Negatif, veto, reddet
          dangerDim: "rgba(255, 69, 58, 0.15)",
          text: "#F5F5F7",
          textMuted: "#86868B",
        },
      },
      fontFamily: {
        display: ['"Playfair Display"', "serif"], // Sinematik başlıklar
        body: ['"Inter"', "sans-serif"],          // Okunabilir gövde
        mono: ['"JetBrains Mono"', "monospace"],  // Veri ve metrikler
      },
      boxShadow: {
        glowGold: "0 0 20px rgba(255, 179, 0, 0.3)",
        glowCyan: "0 0 20px rgba(0, 229, 255, 0.3)",
        glass: "0 8px 32px 0 rgba(0, 0, 0, 0.37)",
      },
      animation: {
        "pulse-slow": "pulse 4s cubic-bezier(0.4, 0, 0.6, 1) infinite",
        "shimmer": "shimmer 2s linear infinite",
      },
      keyframes: {
        shimmer: {
          "0%": { backgroundPosition: "-1000px 0" },
          "100%": { backgroundPosition: "1000px 0" },
        },
      },
    },
```

```
  },
  plugins: [],
};
```

`src/index.css` (Global Stiller ve Fontlar)
```css
@import
url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600&family=JetBrain
s+Mono:wght@400;500&family=Playfair+Display:wght@600;700&display=swap');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  body {
    @apply bg-cinema-bg text-cinema-text font-body antialiased;
    background-image:
      radial-gradient(circle at 15% 50%, rgba(0, 229, 255, 0.03) 0%, transparent 25%),
      radial-gradient(circle at 85% 30%, rgba(255, 179, 0, 0.03) 0%, transparent 25%);
  }
}

@layer components {
  .glass-panel {
    @apply bg-cinema-surface/60 backdrop-blur-xl border border-white/5 rounded-2xl
shadow-glass;
  }

  .glass-card {
    @apply bg-cinema-surface/40 backdrop-blur-md border border-white/5 rounded-xl
transition-all duration-300 hover:bg-cinema-surfaceHover hover:border-white/10;
  }

  .metric-value {
    @apply font-mono text-2xl font-semibold tracking-tight;
  }

  .btn-cinematic {
    @apply px-4 py-2 rounded-lg font-medium transition-all duration-200 flex items-center
justify-center gap-2;
  }

  .btn-primary {
    @apply btn-cinematic bg-cinema-gold text-black hover:bg-amber-400
hover:shadow-glowGold;
  }
```

```
  .btn-ghost {
    @apply btn-cinematic text-cinema-textMuted hover:text-cinema-text hover:bg-white/5;
  }
}

/* Özel Scrollbar */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  @apply bg-transparent;
}
::-webkit-scrollbar-thumb {
  @apply bg-white/10 rounded-full hover:bg-white/20 transition-colors;
}
```

🧩 2. Temel UI Bileşenleri (Yeniden Kullanılabilir)

`src/components/ui/GlassCard.tsx`
```tsx
import React from "react";
import { motion } from "framer-motion";

interface GlassCardProps {
  children: React.ReactNode;
  className?: string;
  glow?: "gold" | "cyan" | "none";
  onClick?: () => void;
}

export const GlassCard: React.FC<GlassCardProps> = ({
  children,
  className = "",
  glow = "none",
  onClick
}) => {
  const glowClass = glow === "gold" ? "shadow-glowGold border-cinema-gold/30" :
                    glow === "cyan" ? "shadow-glowCyan border-cinema-cyan/30" : "";

  return (
    <motion.div
      whileHover={{ scale: 1.02, y: -2 }}
      whileTap={{ scale: 0.98 }}
```

```
      onClick={onClick}
      className={`glass-card ${glowClass} ${className} ${onClick ? "cursor-pointer" : ""}`}
    >
      {children}
    </motion.div>
  );
};
```

🎬 3. Ana Ekran: Sinema Kimliğim (Taste Profile Dashboard)

Bu bileşen, "Bunu nasıl bildin?" hissini veren ana paneldir.

`src/components/taste/TasteProfileDashboard.tsx`
```tsx
import React, { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { GlassCard } from "../ui/GlassCard";
import { TasteMetricDetail } from "./TasteMetricDetail";
// Not: useTasteProfile hook'unun mevcut implementasyonunuzu döndürdüğünü varsayıyoruz.
// import { useTasteProfile } from "../../hooks/useTasteProfile";

// Örnek veri (Gerçek hook bağlandığında burası dinamik olacak)
const MOCK_METRICS = [
  { metricId: "genre.scifi", affinity: 85, status: "directional", evidenceCount: 12, definition: "Bilim kurgu temalı içerikler" },
  { metricId: "mood.melancholic", affinity: 72, status: "directional", evidenceCount: 8, definition: "Melankolik, hüzünlü atmosfer" },
  { metricId: "concept.identity", affinity: 60, status: "mixed", evidenceCount: 5, definition: "Kimlik, benlik temaları" },
  { metricId: "genre.romance", affinity: -40, status: "directional", evidenceCount: 6, definition: "Romantik türü içerikler" },
  { metricId: "setting.space", affinity: 0, status: "unknown", evidenceCount: 0, definition: "Uzay, uzay gemisi ortamı" },
];

export const TasteProfileDashboard: React.FC = () => {
  const [selectedMetric, setSelectedMetric] = useState<string | null>(null);
  // const { profile, loading } = useTasteProfile(); // Gerçek entegrasyon

  const getStatusColor = (status: string) => {
    switch (status) {
      case "directional": return "text-cinema-gold";
      case "mixed": return "text-cinema-cyan";
      case "unknown": return "text-cinema-textMuted";
```

```
    default: return "text-cinema-text";
  }
 };

 const getStatusLabel = (status: string) => {
  switch (status) {
   case "directional": return "Güçlü Eğilim";
   case "mixed": return "Karışık Kanıt";
   case "unknown": return "Veri Yetersiz";
   default: return status;
  }
 };

 return (
  <div className="max-w-6xl mx-auto p-6 space-y-8">
   {/* Hero Bölümü */}
   <div className="text-center space-y-4 mb-12">
    <h1 className="font-display text-4xl md:text-5xl font-bold text-cinema-text">
     Sinema <span className="text-cinema-gold">Kimliğim</span>
    </h1>
    <p className="text-cinema-textMuted max-w-2xl mx-auto text-lg">
     İzleme geçmişinizden yola çıkarak, zevkinizin matematiksel ve semantik haritasını
çıkardık.
     Her bir değerin arkasında somut kanıtlar var.
    </p>
   </div>

   {/* Metrik Izgarası (Constellation Map hissi) */}
   <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
    {MOCK_METRICS.map((metric) => (
     <GlassCard
      key={metric.metricId}
      glow={metric.affinity > 70 ? "gold" : metric.affinity < -20 ? "none" : "cyan"}
      onClick={() => setSelectedMetric(metric.metricId)}
      className="p-6 flex flex-col justify-between h-40 group"
     >
      <div className="flex justify-between items-start">
       <span className="font-mono text-xs text-cinema-textMuted uppercase
tracking-wider">
        {metric.metricId.split(".")[0]}
       </span>
       <span className={`text-xs font-medium px-2 py-1 rounded-full bg-white/5
${getStatusColor(metric.status)}`}>
        {getStatusLabel(metric.status)}
       </span>
      </div>

      <div>
```

```
            <h3 className="font-display text-xl font-semibold text-cinema-text mb-1
group-hover:text-cinema-gold transition-colors">
              {metric.metricId.split(".")[1].replace("_", " ")}
            </h3>
            <div className="flex items-baseline gap-2">
              <span className={`metric-value ${metric.affinity > 0 ? "text-cinema-gold" :
metric.affinity < 0 ? "text-cinema-danger" : "text-cinema-textMuted"}`}>
                {metric.affinity > 0 ? "+" : ""}{metric.affinity}
              </span>
              <span className="text-xs text-cinema-textMuted">
                ({metric.evidenceCount} kanıt)
              </span>
            </div>
          </div>
        </GlassCard>
      ))}
    </div>

    {/* Detay Modalı */}
    <AnimatePresence>
      {selectedMetric && (
        <TasteMetricDetail
          metricId={selectedMetric}
          onClose={() => setSelectedMetric(null)}
        />
      )}
    </AnimatePresence>
  </div>
 );
};
```

🔍 4. Metrik Detay ve Kanıt Listesi (TasteMetricDetail)

Bir metriğe tıklandığında açılan, cam efektli, "Bunu nasıl bildin?" sorusunu yanıtlayan modal.

`src/components/taste/TasteMetricDetail.tsx`
```tsx
import React from "react";
import { motion } from "framer-motion";
import { GlassCard } from "../ui/GlassCard";
import { TastePreferenceEditor } from "./TastePreferenceEditor";
import { X, Film, CheckCircle, XCircle, AlertCircle } from "lucide-react";

interface TasteMetricDetailProps {
  metricId: string;
```

```
  onClose: () => void;
}

// Örnek kanıt verisi
const MOCK_EVIDENCE = {
  definition: "Bilim kurgu temalı içerikler",
  affinity: 85,
  status: "directional",
  positive: [
    { title: "Interstellar", year: 2014, contribution: 0.92, mediaType: "movie" },
    { title: "Blade Runner 2049", year: 2017, contribution: 0.88, mediaType: "movie" },
    { title: "Dune", year: 2021, contribution: 0.85, mediaType: "movie" },
  ],
  negative: [
    { title: "Transformers", year: 2007, contribution: -0.60, mediaType: "movie" },
  ],
};

export const TasteMetricDetail: React.FC<TasteMetricDetailProps> = ({ metricId, onClose })
=> {
  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      exit={{ opacity: 0 }}
      className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/80
backdrop-blur-sm"
      onClick={onClose}
    >
      <motion.div
        initial={{ scale: 0.95, y: 20 }}
        animate={{ scale: 1, y: 0 }}
        exit={{ scale: 0.95, y: 20 }}
        className="glass-panel w-full max-w-2xl max-h-[90vh] overflow-y-auto"
        onClick={(e) => e.stopPropagation()}
      >
        {/* Header */}
        <div className="sticky top-0 z-10 bg-cinema-surface/90 backdrop-blur-xl border-b
border-white/5 p-6 flex justify-between items-start">
          <div>
            <span className="font-mono text-xs text-cinema-cyan uppercase tracking-widest">
              {metricId.split(".")[0]} METRIC
            </span>
            <h2 className="font-display text-3xl font-bold text-cinema-text mt-1">
              {metricId.split(".")[1].replace("_", " ")}
            </h2>
            <p className="text-cinema-textMuted text-sm
mt-1">{MOCK_EVIDENCE.definition}</p>
```

```
        </div>
        <button
          onClick={onClose}
          className="p-2 rounded-full hover:bg-white/10 text-cinema-textMuted
hover:text-cinema-text transition-colors"
        >
          <X size={24} />
        </button>
       </div>

       <div className="p-6 space-y-8">
        {/* Ana Skor ve Editör */}
        <div className="flex flex-col md:flex-row gap-6 items-start">
          <div className="glass-card p-6 flex-1 w-full text-center">
            <div className="text-cinema-textMuted text-sm mb-2">Otomatik Hesaplanan
Afinite</div>
            <div className="metric-value text-5xl text-cinema-gold mb-2">
              +{MOCK_EVIDENCE.affinity}
            </div>
            <div className="flex items-center justify-center gap-2 text-sm
text-cinema-textMuted">
              <CheckCircle size={16} className="text-cinema-gold" />
              <span>Güçlü Kanıt ({MOCK_EVIDENCE.positive.length +
MOCK_EVIDENCE.negative.length} içerik)</span>
            </div>
          </div>

          <div className="flex-1 w-full">
            <TastePreferenceEditor metricId={metricId}
currentValue={MOCK_EVIDENCE.affinity} />
          </div>
        </div>

        {/* Kanıt Listeleri */}
        <div className="space-y-6">
          {/* Olumlu Kanıtlar */}
          <div>
            <h3 className="font-display text-lg font-semibold text-cinema-text mb-3 flex
items-center gap-2">
              <CheckCircle size={18} className="text-cinema-gold" />
              Tercihini Güçlendiren İçerikler
            </h3>
            <div className="space-y-2">
              {MOCK_EVIDENCE.positive.map((item, idx) => (
                <GlassCard key={idx} className="p-3 flex justify-between items-center">
                  <div className="flex items-center gap-3">
                    <div className="w-10 h-14 bg-cinema-surfaceHover rounded flex
items-center justify-center">
```

```
                  <Film size={20} className="text-cinema-textMuted" />
                </div>
                <div>
                  <div className="font-medium text-cinema-text">{item.title}</div>
                  <div className="text-xs text-cinema-textMuted">{item.year} ·
{item.mediaType}</div>
                </div>
              </div>
              <div className="font-mono text-cinema-gold font-semibold">
                +{(item.contribution * 100).toFixed(0)}
              </div>
            </GlassCard>
          ))}
        </div>
      </div>

      {/* Olumsuz Kanıtlar */}
      {MOCK_EVIDENCE.negative.length > 0 && (
        <div>
          <h3 className="font-display text-lg font-semibold text-cinema-text mb-3 flex
items-center gap-2">
            <XCircle size={18} className="text-cinema-danger" />
            Tercihini Zayıflatan İçerikler
          </h3>
          <div className="space-y-2">
            {MOCK_EVIDENCE.negative.map((item, idx) => (
              <GlassCard key={idx} className="p-3 flex justify-between items-center
border-cinema-danger/10">
                <div className="flex items-center gap-3">
                  <div className="w-10 h-14 bg-cinema-surfaceHover rounded flex
items-center justify-center">
                    <Film size={20} className="text-cinema-textMuted" />
                  </div>
                  <div>
                    <div className="font-medium text-cinema-text">{item.title}</div>
                    <div className="text-xs text-cinema-textMuted">{item.year} ·
{item.mediaType}</div>
                  </div>
                </div>
                <div className="font-mono text-cinema-danger font-semibold">
                  {(item.contribution * 100).toFixed(0)}
                </div>
              </GlassCard>
            ))}
          </div>
        </div>
      )}
    </div>
```

```
        {/* Uyarı / Bilgi Notu */}
        <div className="glass-card p-4 border-cinema-cyan/20 bg-cinema-cyanDim/10 flex
gap-3">
          <AlertCircle size={20} className="text-cinema-cyan shrink-0 mt-0.5" />
          <p className="text-sm text-cinema-textMuted leading-relaxed">
            <span className="text-cinema-text font-medium">Bunu nasıl bildin?</span> Bu
skor,
            yukarıdaki içeriklere verdiğiniz puanların, bu metriklerle olan matematiksel
çarpımının
            zaman ağırlıklı ortalamasıdır. Yanlış bir etiket olduğunu düşünüyorsanız,
            yukarıdaki "Manuel Kontrol" panelini kullanabilirsiniz.
          </p>
          {/* Not: Gerçek uygulamada buraya "Bu film aslında X değil" itiraz butonu eklenecek
(V1.5) */}
        </div>
      </div>
    </motion.div>
  </motion.div>
 );
};
```

🎛️ 5. Manuel Kontrol Paneli (TastePreferenceEditor)

Kullanıcının "Yanlışımı düzeltmene izin veriyorum" vaadini gerçekleştiren bileşen.

`src/components/taste/TastePreferenceEditor.tsx`
```tsx
import React, { useState } from "react";
import { motion } from "framer-motion";
import { Settings2, RotateCcw, ThumbsUp, ThumbsDown, Ban } from "lucide-react";

interface TastePreferenceEditorProps {
  metricId: string;
  currentValue: number;
}

export const TastePreferenceEditor: React.FC<TastePreferenceEditorProps> = ({
  metricId,
  currentValue
}) => {
  const [mode, setMode] = useState<"auto" | "boost" | "suppress" | "exclude">("auto");

  const handleModeChange = (newMode: typeof mode) => {
    setMode(newMode);
```

```
    // TODO: Burada taste-api.postManualOverride(metricId, newValue, newMode) çağrısı
yapılacak
    console.log(`Manuel kontrol güncellendi: ${metricId} -> ${newMode}`);
  };

  return (
    <div className="glass-card p-5 w-full">
      <div className="flex items-center gap-2 mb-4 text-cinema-textMuted">
        <Settings2 size={16} />
        <span className="text-xs font-semibold uppercase tracking-wider">Manuel
Kontrol</span>
      </div>

      <div className="grid grid-cols-2 gap-2">
        <button
          onClick={() => handleModeChange("auto")}
          className={`p-3 rounded-lg text-sm font-medium transition-all flex flex-col
items-center gap-1 ${
            mode === "auto"
              ? "bg-cinema-cyan/20 text-cinema-cyan border border-cinema-cyan/30"
              : "bg-white/5 text-cinema-textMuted hover:bg-white/10"
          }`}
        >
          <RotateCcw size={18} />
          <span>Otomatik</span>
          <span className="text-[10px] opacity-70">({currentValue > 0 ? "+" :
""}{currentValue})</span>
        </button>

        <button
          onClick={() => handleModeChange("boost")}
          className={`p-3 rounded-lg text-sm font-medium transition-all flex flex-col
items-center gap-1 ${
            mode === "boost"
              ? "bg-cinema-gold/20 text-cinema-gold border border-cinema-gold/30"
              : "bg-white/5 text-cinema-textMuted hover:bg-white/10"
          }`}
        >
          <ThumbsUp size={18} />
          <span>Güçlendir</span>
        </button>

        <button
          onClick={() => handleModeChange("suppress")}
          className={`p-3 rounded-lg text-sm font-medium transition-all flex flex-col
items-center gap-1 ${
            mode === "suppress"
              ? "bg-cinema-danger/20 text-cinema-danger border border-cinema-danger/30"
```

```
            : "bg-white/5 text-cinema-textMuted hover:bg-white/10"
          }`}
        >
          <ThumbsDown size={18} />
          <span>Azalt</span>
        </button>

        <button
          onClick={() => handleModeChange("exclude")}
          className={`p-3 rounded-lg text-sm font-medium transition-all flex flex-col
items-center gap-1 ${
            mode === "exclude"
              ? "bg-white/10 text-cinema-text border border-white/20"
              : "bg-white/5 text-cinema-textMuted hover:bg-white/10"
          }`}
        >
          <Ban size={18} />
          <span>İlgisiz</span>
        </button>
      </div>

      {mode !== "auto" && (
        <motion.div
          initial={{ opacity: 0, height: 0 }}
          animate={{ opacity: 1, height: "auto" }}
          className="mt-4 pt-4 border-t border-white/5"
        >
          <p className="text-xs text-cinema-textMuted text-center">
            {mode === "boost" && "Sistem bu metriği otomatik değerden daha yüksek ağırlıkla
hesaplayacak."}
            {mode === "suppress" && "Sistem bu metriğin etkisini azaltacak, ancak tamamen
yok saymayacak."}
            {mode === "exclude" && "Bu metrik, size içerik önerirken tamamen dikkate
alınmayacak."}
          </p>
          <button
            onClick={() => handleModeChange("auto")}
            className="mt-3 w-full py-2 text-xs text-cinema-cyan hover:text-cinema-text
transition-colors"
          >
            Değişikliği Geri Al (Otomatiğe Dön)
          </button>
        </motion.div>
      )}
    </div>
  );
};
```

🚀 6. Entegrasyon ve Sonraki Adımlar

Bu kodları projenize ekledikten sonra yapmanız gerekenler:

1. **Bağımlılıkları Yükleyin:**
   ```bash
   npm install framer-motion lucide-react
   ```
2. **Fontları Doğrulayın:** `index.css` içindeki `@import` satırının en üstte olduğundan emin olun.
3. **Hook Entegrasyonu:** `TasteProfileDashboard.tsx` içindeki `MOCK_METRICS` verisini, daha önce oluşturduğumuz `useTasteProfile()` hook'undan gelen gerçek `profile.metrics` verisiyle değiştirin.
4. **App.tsx'e Bağlayın:** Mevcut `ProfileStatsTab.tsx` veya ilgili sekme bileşeninizin içinde `<TasteProfileDashboard />` bileşenini render edin.